Universidad de San Carlos de Guatemala
Escuela de Ingeniería en Ciencias y Sistemas, Facultad de Ingeniería
Introducción a la programación y computación 2, 1er. Semestre 2026

Sistema de Control Bancario

202307764 – Julio Eduardo Gustavo Alvarado Ramirez

Resumen

Este ensayo expone el desarrollo de un Sistema de Control Bancario integral, diseñado como solución tecnológica ante la creciente necesidad de automatización financiera en el contexto guatemalteco. El tema destaca por su vigencia internacional en la digitalización de servicios, permitiendo una gestión transparente de créditos y débitos. Durante la implementación, se adoptaron posturas técnicas centradas en la arquitectura de servicios desacoplados, utilizando una API en C# para la lógica de negocios y Razor Pages para la interfaz de usuario. El impacto técnico es significativo al garantizar la integridad de los datos mediante el procesamiento de archivos XML y el uso de expresiones regulares. Económicamente, el sistema optimiza la conciliación bancaria, mientras que socialmente empodera al cliente con acceso inmediato a su estado de cuenta histórico. Se concluye que la modularidad y la persistencia de datos son pilares fundamentales para el éxito de cualquier software financiero, asegurando que la tecnología no solo sea funcional, sino también robusta, escalable y capaz de responder a las exigencias de transparencia y precisión del mercado actual.

Palabras Clave

1. Control bancario
2. C#
3. Procesamiento XML
4. Razor Pages
5. Persistencia

Abstract

This essay presents the development of a comprehensive Banking Control System, designed as a technological solution to the growing need for financial automation within the Guatemalan context. The subject stands out for its international relevance in the digitalization of services, enabling the transparent management of credits and debits. During implementation, technical approaches focused on a decoupled service architecture were adopted, utilizing a C# API for business logic and Razor Pages for the user interface. The technical impact is significant, ensuring data integrity through XML file processing and the use of regular expressions. Economically, the system optimizes bank reconciliation, while socially, it empowers clients with immediate access to their historical account statements. It is concluded that modularity and data persistence are fundamental pillars for the success of any financial software, ensuring that the technology is not only functional but also robust, scalable, and capable of responding to the transparency and accuracy demands of the current market..

Keywords

1. *Banking control*
2. *C#*
3. *XML Processing*
4. *Razor Pages*
5. *Persistence*

Introducción

La evolución tecnológica ha transformado radicalmente la forma en que las instituciones operan, migrando de procesos manuales propensos al error hacia plataformas digitales altamente automatizadas. En este contexto de transformación digital, el desarrollo y la implementación de un Sistema de Control Bancario robusto se vuelven una necesidad trascendental para garantizar la eficiencia, la seguridad y la transparencia en la gestión de transacciones financieras. El presente ensayo aborda la construcción de una solución web integral, analizando cómo las buenas prácticas de la ingeniería de software responden de manera efectiva a las rigurosas exigencias del sector económico. Desde una perspectiva técnica, el análisis se fundamenta en la adopción de una arquitectura cliente-servidor y el principio de separación de responsabilidades. Se explora el uso de un backend desarrollado en C# bajo un enfoque de API REST, encargado de la lógica de negocios y las validaciones mediante expresiones regulares, operando en conjunto con un frontend dinámico construido con Razor Pages. Además, el documento toma como base teórica la vital importancia de la persistencia de datos. El uso de archivos XML como medio de almacenamiento físico demuestra cómo la información financiera (clientes, bancos, facturas y pagos) debe sobrevivir a la volatilidad de la memoria RAM para asegurar la integridad de los estados de cuenta a lo largo del tiempo. El propósito principal de este ensayo es desglosar la implementación y el impacto de este sistema, brindando al lector una visión clara de los desafíos técnicos resueltos y los beneficios operativos de una arquitectura modular. A lo largo del desarrollo del texto, se buscará dar respuesta a interrogantes clave: ¿Cómo puede un sistema web asegurar la precisión matemática y la consistencia en el cálculo de saldos bancarios? ¿De qué manera la validación estricta de archivos de configuración optimiza la toma de decisiones financieras? En definitiva, este trabajo busca familiarizar al lector con la intersección entre la programación avanzada y las finanzas, demostrando que la correcta estructuración de los datos y el diseño de interfaces amigables son pilares fundamentales para el éxito de cualquier plataforma de gestión bancaria en la actualidad. (Cormen, 2009)

Arquitectura Cliente-Servidor y Separación de Responsabilidades

El diseño y construcción de un Sistema de Control Bancario moderno exige la adopción de paradigmas de programación que garanticen escalabilidad, seguridad y eficiencia. En este marco referencial, el principio fundamental aplicado es la separación de responsabilidades a través de una arquitectura cliente-servidor, la cual divide la aplicación en dos grandes capas: el Frontend y el Backend. Esta dicotomía tecnológica no solo facilita el mantenimiento del código, sino que optimiza el rendimiento general del sistema. (Freeman, 2022)

Capas de la Aplicación: Backend Funcional y Frontend Dinámico

Por un lado, el Backend, desarrollado bajo el ecosistema de C# .NET, asume el rol del cerebro computacional. Es el encargado de procesar la lógica de negocios, ejecutar cálculos matemáticos precisos para la conciliación de saldos y aplicar estrictas validaciones mediante el uso de expresiones regulares. Por otro lado, el Frontend, construido con la tecnología Razor Pages y estilizado mediante marcos de trabajo como Bootstrap, actúa como la capa de presentación. Su propósito es ofrecer al usuario final (en este caso, administradores financieros) una interfaz gráfica intuitiva, ergonómica y capaz de renderizar dashboards complejos, como las gráficas dinámicas de ingresos bancarios. (Gansner, 2000)

Intercomunicación mediante APIs y Seguridad HTTPS

Para que estas dos capas independientes interactúen, el ecosistema se apoya en el uso de APIs (Application Programming Interfaces). La API RESTful diseñada para este proyecto actúa como un puente de comunicación estandarizado, recibiendo peticiones HTTP desde la interfaz y devolviendo respuestas estructuradas. Sin embargo, dada la naturaleza crítica de la información procesada —que incluye el NIT de clientes, historiales de pago y montos de facturación—, la seguridad en este canal de comunicación es una prioridad ineludible. Es aquí donde el protocolo HTTPS (Hypertext Transfer Protocol Secure) cobra vital importancia disciplinar. La adopción de HTTPS garantiza que los datos viajen encriptados mediante certificados SSL/TLS, protegiendo al sistema de vulnerabilidades comunes como la interceptación de paquetes o ataques de intermediario (Man-in-the-Middle). (Harold, 2004)

El Desafío de la Memoria Volátil y la Persistencia de Datos

Más allá de la comunicación en tránsito, el desafío técnico más complejo en un entorno de transacciones financieras es la retención de la información. La memoria de la computadora (RAM) es volátil por naturaleza; cualquier interrupción del servidor resultaría en la pérdida total del estado de cuenta de los clientes. Para mitigar este riesgo, el sistema implementa una sólida estrategia de persistencia de datos. La persistencia asegura que la información trascienda el ciclo de vida del programa en ejecución, almacenándose de forma permanente en un medio físico, como un disco duro.

Implementación de Almacenamiento Físico mediante XML

Para este sistema en particular, la persistencia se materializa mediante el uso del lenguaje de marcado XML (eXtensible Markup Language). La elección de XML no es casual; su estructura jerárquica de etiquetas proporciona un formato de intercambio de datos universalmente aceptado, legible tanto para máquinas como para humanos. Cada vez que el Backend procesa una nueva carga de transacciones, el patrón de diseño Singleton actualiza el estado en memoria e inmediatamente sincroniza esta información sobrescribiendo los archivos físicos de configuración y transacciones. De esta manera, se garantiza la integridad y la disponibilidad de los datos financieros, permitiendo que el sistema pueda detenerse, reiniciarse y recuperar su estado exacto sin inconsistencias.

Sinergia Tecnológica y Resiliencia Financiera

En síntesis, la orquestación de estas tecnologías trasciende la simple escritura de código; representa la aplicación de buenas prácticas de la ingeniería de software para resolver problemas contextuales reales. La sinergia entre un Frontend amigable, un Backend inteligente comunicado por APIs seguras, y un mecanismo de persistencia basado en XML, da como resultado un sistema bancario resiliente y preparado para las exigencias de la auditoría y el control financiero. (Joyanes Aguilar, 2008)

Conclusiones

En conclusión, el desarrollo del Sistema de Control Bancario demuestra que la ingeniería de software aplicada al sector financiero debe priorizar invariablemente la seguridad, la modularidad y la integridad de los datos. El análisis del proyecto evidencia que la implementación de una arquitectura cliente-servidor, donde se separa estrictamente la lógica de negocios (Backend) de la capa de presentación visual (Frontend), se consolida no solo como una preferencia de diseño, sino como un estándar tecnológico indispensable. Esta postura garantiza que la plataforma sea escalable, segura y capaz de adaptarse a futuras expansiones sin comprometer su núcleo operativo.

Uno de los aportes más significativos de este estudio es la ratificación de la persistencia de datos como el pilar absoluto de la confiabilidad bancaria. La

transición de un estado volátil en memoria RAM hacia un almacenamiento físico y estructurado mediante archivos XML, subraya la premisa de que un software financiero carece de valor si no puede proteger la información ante interrupciones. Se concluye que el manejo automatizado y persistente de historiales de pago, facturas y saldos es lo que verdaderamente transforma un simple programa de cálculo en una herramienta de auditoría y conciliación profesional.

Finalmente, a partir de la implementación realizada, se propone que el éxito de la transformación digital en la banca moderna radica en la sinergia entre interfaces centradas en el usuario y comunicaciones encriptadas. Como autor, la postura central que se desea transmitir es que la verdadera innovación tecnológica no reside únicamente en la escritura de código complejo, sino en la capacidad de construir ecosistemas digitales resilientes. Al automatizar estos procesos, no solo se optimizan los recursos económicos de la institución, sino que se otorga total transparencia y certeza al cliente final, cumpliendo así el propósito fundamental de la tecnología: resolver problemas reales con eficiencia y precisión.

Bibliografía

Cormen, T. H. (2009). *Introduction to Algorithms (3ra ed.). MIT Press.*

Freeman, A. (2022). *Pro ASP.NET Core 6: Develop Cloud-Ready Web Applications Using MVC, Blazor, and Razor Pages (9na ed.). Apress.*

Gansner, E. R. (2000). *An open graph visualization system and its applications to software engineering. Software: Practice and Experience, 30(11), 1203-1233.*

Harold, E. R. (2004). *XML in a Nutshell (3ra ed.). O'Reilly Media.*

Joyanes Aguilar, L. (2008). *Fundamentos de programación: Algoritmos, estructura de datos y objetos (4ta ed.). McGraw-Hill.*

Anexos.

Diagrama de Clases

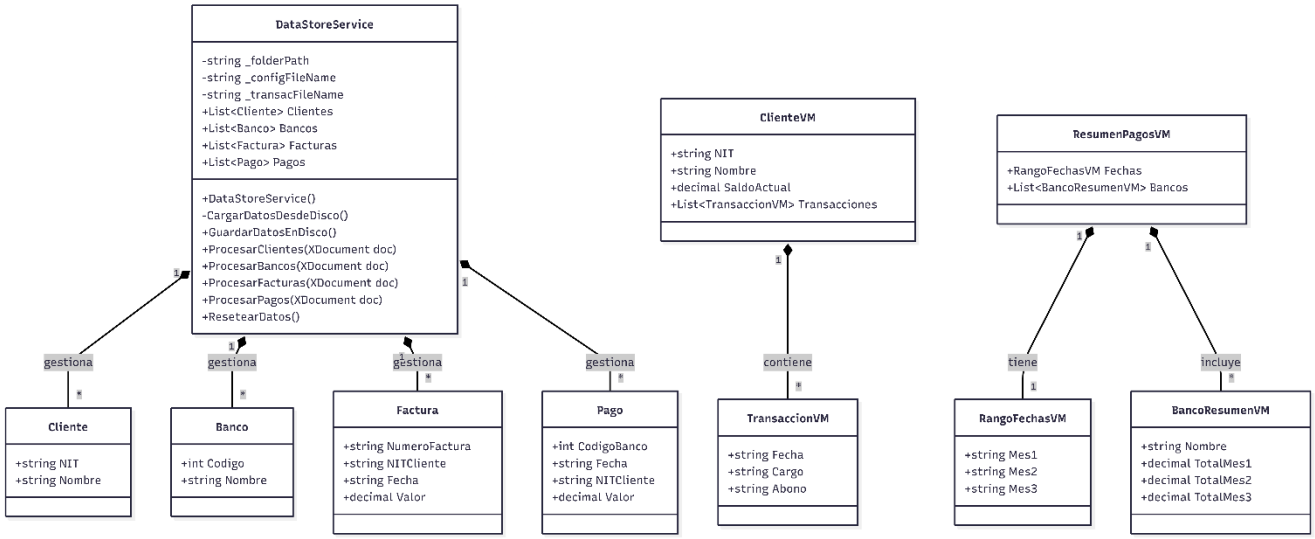

Figura 1. Diagrama de clases.
Fuente: Elaboración Propia

Diagrama Entidad-Relación

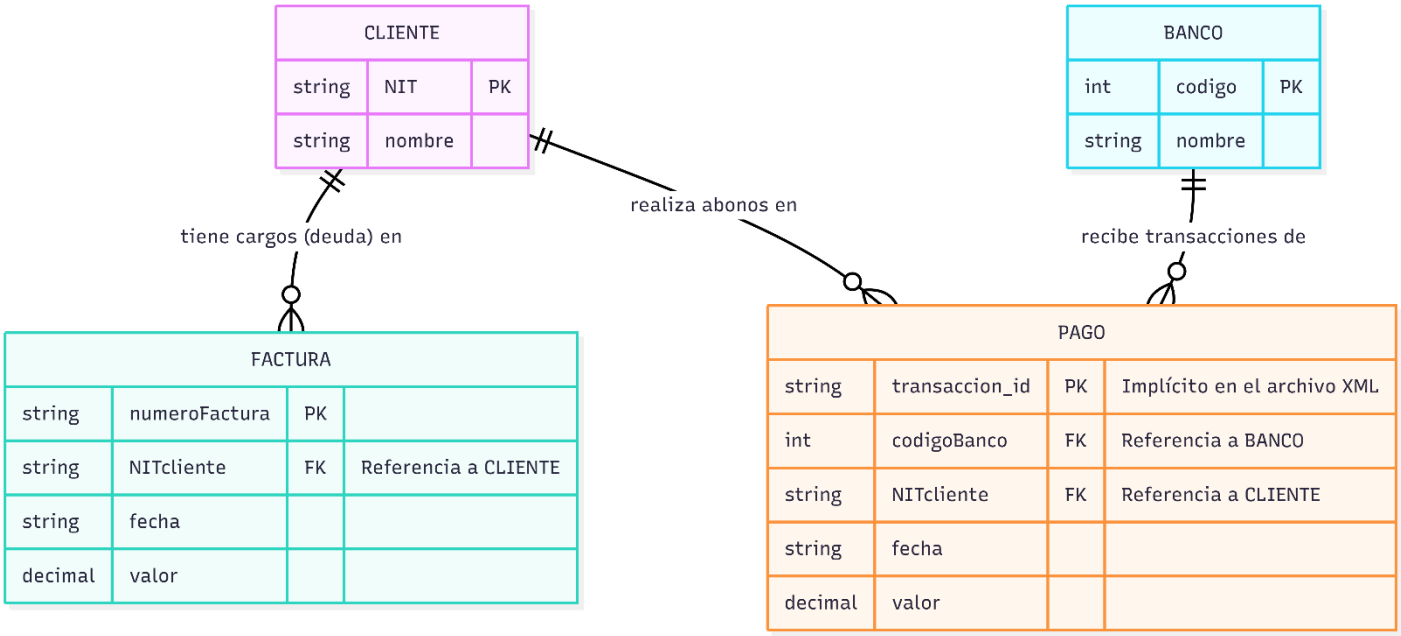

Figura 2. Diagrama Entidad-Relación
Fuente: Elaboración Propia

Diagrama de Actividades

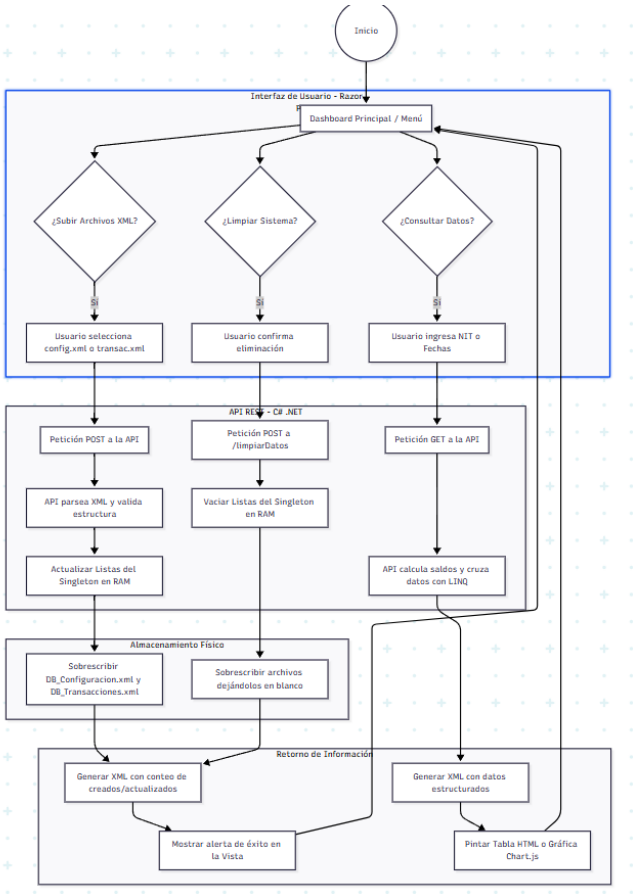

Figura 3. Diagrama de actividades
Fuente: Elaboración Propia